

OPERATION COLD IRON: Authenticated XChaCha20-Poly1305 Cold-Chain Telemetry and the Two Unauthenticated Control Surfaces of an RP2350 Node

Kevin Thomas
George Mason University
Fairfax, VA, USA
kthoma60@gmu.edu

Abstract: Cold-chain and agricultural monitoring trusts a cheap endpoint to tell the truth about temperature. The OPERATION COLD IRON build is a bare-metal RP2350 depot node that samples a DHT11, renders a 1602 LCD, annunciates state with red, yellow, and green LEDs, takes an operator acknowledge button, drives a cold-store damper with an SG90 servo, decodes a VS1838B NEC infrared remote, and carries telemetry over an RYLR998 LoRa link. Two control surfaces on the node are unauthenticated: the LoRa sender field and the infrared eye. Both are demonstrated and then closed where cryptography can reach. Telemetry is sealed end to end with XChaCha20-Poly1305 (RFC 8439 ChaCha20 and Poly1305 with an HChaCha20 subkey) keyed through Argon2id (RFC 9106, profile t=3, p=1, m=64 blocks), implemented in-repo with no third-party code and tested against published vectors. Each frame is carried as a lowercase hex envelope of nonce, ciphertext, and tag, with the node identifier bound as associated data. The hub gateway authenticates before it parses, so the spoofing client that once injected forged readings now fails at the tag and its JSON is never logged. We document the peripheral set, the wire and envelope formats, the cryptographic design and the rationale for ChaCha20 over AES on this silicon, the gateway verification path, and an honest threat model in which the lab key derivation, the committed secrets, and the unauthenticated infrared path remain explicit limitations.

Index Terms: RP2350, DHT11, LoRa, RYLR998, XChaCha20-Poly1305, Argon2id, authenticated encryption, NEC infrared replay, sender spoofing, embedded firmware.

I. Introduction

Low-power wireless sensing for perishable supply chains and small-plot agriculture has a classic deployment tension: the endpoint must be cheap, open, and simple enough for classroom tinkering, yet the telemetry it emits must survive inspection by a curious adversary. Sub-GHz modules such as the RYLR998 solve the cost and simplicity problem by shipping a fixed serial AT interface: AT+SEND to transmit, +RCV reports arriving as CRLF-delimited text lines. The implicit security assumption of this interface is benign: whoever holds the radio controls the network.

That assumption is exactly what this project attacks and then repairs. Any radio on the same band and key set can be pointed at a victim node address with AT+ADDRESS, and a receiver that trusts the sender field has no way to tell the difference. The OPERATION COLD IRON build makes the attack legible for the classroom and then makes it fail. A legitimate firmware node streams sealed telemetry, a hub gateway authenticates and logs it, and a spoofing client demonstrates that a forged frozen-temperature frame dies at the authentication tag instead of triggering the hub actuator reply.

The node is more than a sensor and a radio. It carries the full Embedded Hacking peripheral kit: three status LEDs, an acknowledge button, an SG90 damper servo,

and a VS1838B infrared receiver. Together they turn a data-integrity bug into a physical one. The infrared eye is a second unauthenticated control surface: any NEC remote can vent or seal the cold-store damper, and a captured command can be replayed. The servo is the physical consequence, the thing that makes the missing authentication matter to a person rather than to a log file.

The design objective is a small, fully tested, reproducible artifact that documents both the legitimate authenticated wire protocol and the exact fields and surfaces a hardening effort must protect.

I.I. Contributions

This paper provides the following concrete contributions:

- A bare-metal RP2350 firmware that drives the full peripheral set: DHT11 one-wire sampling, 1602 LCD rendering over I2C, a red, yellow, and green status annunciator, a debounced acknowledge button, an SG90 damper servo, a VS1838B NEC infrared decoder, and RYLR998 telemetry with a declared-length payload parser.
- A demonstration of two unauthenticated control surfaces, a LoRa sender spoof and an NEC infrared replay, with the damper servo as the physical consequence of the second.
- An in-repo, third-party-free cryptographic layer: Argon2id key derivation (RFC 9106) and XChaCha20-Poly1305 authenticated encryption (RFC 8439 with an HChaCha20 subkey), sealed per frame into a lowercase hex envelope with the node identifier bound as associated data.
- An authenticated hub gateway that verifies the tag before parsing, logs authenticated and rejected frames distinctly, and never treats forged JSON as truth, plus a spoofing client whose forged frames are now rejected.
- A corpus-aligned packet artifact contract (`packet_artifact.json` / `packet_artifact.h`) with a build-time staleness guardrail.
- An 89-case native test suite reaching 100% line coverage of every owned firmware module, including all cryptographic primitives, which are checked against published RFC test vectors.
- A threat model that states explicitly what the lab profile does and does not protect.

II. Related Work

Environmental monitoring is a mature embedded application area. The DHT11 one-wire sensor [1] and its checksummed 40-bit response format are broadly documented in hobbyist literature and microcontroller datasheets; the Protocol section below formalizes the edge-timing decoder we actually test.

Sub-GHz serial AT radios have seen prior security scrutiny: several analyses [2, 3] demonstrate that RYLR-class modules carry no per-frame authentication, that AT+ADDRESS, band, and key configuration are shipped in cleartext, and that firmware misconfiguration of the serial interface is a common root cause of exposure. The cold-chain monitoring literature [4] focuses on temperature logging and out-of-range alerting without addressing message origin.

Authenticated approaches are standardized and well understood. LoRaWAN applies AES-128 application-layer security [5]; the AEAD construction we use follows the ChaCha20-Poly1305 standard [7], and the key derivation follows the Argon2 specification [8]. In contrast with a full network stack, the OPERATION COLD IRON classroom scope keeps the authenticated envelope deliberately small and inspectable so the failure mode is observable and testable. This mirrors the pedagogical use of intentionally vulnerable firmware exercises in reverse-engineering coursework [6], except that here the vulnerability is demonstrated and then closed on the same wire.

III. System Model

The system consists of four roles:

- **Node (RP2350 firmware):** samples the DHT11, drives the LEDs, the LCD, the damper servo, and the infrared decoder, and emits sealed AT+SEND telemetry every five seconds.
- **Hub (gateway):** listens on the instructor serial port, authenticates and logs every +RCV frame to CSV, and replies to the authenticated sender when a reported temperature breaches the freeze threshold.
- **Edge simulator:** a laptop process that behaves like an additional node, sealing frames with the same field key.
- **Attacker:** a laptop process that either points AT+ADDRESS at a victim node and injects forged

telemetry, or replays a captured NEC infrared command at the damper.

Let $A \in \{0,1\}^{16}$ be the LoRa node address, L the declared payload byte length, and C the ASCII payload. The unauthenticated wire frame is:

+RCV=, A , ,, L , ,, C , ,, rssi, ,, snr, CRLF

Because C may itself contain commas (the JSON body separates fields with ", "), the receiver must slice by declared length rather than count delimiters. The payload occupies exactly L characters after the second comma, and the tail fields follow the $L+1$ -th character.

III.I. Hardware Configuration

The classroom node is a Pico 2 (RP2350) carrying the full Embedded Hacking kit. The pin map is fixed in `include/cold_chain_monitor.h` and enforced by the native test suite:

Signal	RP2350 GPIO
DHT11 data (one-wire)	GP4
1602 LCD SDA (I2C1)	GP2
1602 LCD SCL (I2C1)	GP3
RYLR998 RX (UART1 TX)	GP8
RYLR998 TX (UART1 RX)	GP9
Infrared receiver (VS1838B)	GP5
Damper servo (SG90 PWM)	GP14
Acknowledge button	GP15
Red breach LED	GP16
Yellow warning LED	GP17
Green nominal LED	GP18
Onboard heartbeat LED	GP25

The LCD backpack uses the PCF8574 at 7-bit address 0×27 . The servo runs from a 50 Hz PWM output with a 1000 uF bulk capacitor on the 5 V rail to absorb the stall current when the damper moves. At boot the node programs its own transceiver (`AT+ADDRESS=7`, `AT+NETWORKID=18`) and the hub gateway programs the receiver (`AT+ADDRESS=1`, `AT+NETWORKID=18`) before logging, so telemetry is only delivered between radios that share the network identifier.

III.II. Two Unauthenticated Control Surfaces

Two paths into the node carry no per-message authentication:

- **LoRa telemetry.** The sender address in every +RCV frame is reported by the radio and is trivially forged with `AT+ADDRESS`.
- **Infrared maintenance remote.** The VS1838B receiver decodes any NEC frame it sees. There is no challenge, no rolling code, and no secret, so a captured vent or seal command can be replayed verbatim.

Both are treated as adversarial in this paper. The cryptographic layer below closes the first surface. The second remains open by design because the hardware in the kit has no secure key store reachable from an unauthenticated optical receiver; it is documented as a limitation rather than papered over.

III.III. Sender-Attribution Trust

Before authentication, the hub records the sender as reported by the radio with no verification:

$\text{sender}(A) \rightarrow \text{sender as claimed on the wire}$

This is the cryptographic gap the exercise exploits. Any radio reconfigured to address $A := A_{\text{victim}}$ produces frames that a naive hub attributes to the victim. The sealed envelope below replaces that trust with a keyed tag that the attacker cannot reproduce.

IV. Wire Protocol

The node transmits a fixed-shape JSON telemetry body:

```
{"n":7,"s":12,"t":235,"h":610}
```

with n the node id, s the monotonic sequence number, t the temperature in tenths of a degree Celsius, and h the relative humidity in tenths of a percent. Integer-tenths are used everywhere to avoid floating point on the RP2350 and to keep frames compact.

The body is written into a fixed 48-byte artifact buffer but only its actual length is sealed and transmitted. That body is then wrapped by the envelope codec, so the on-wire command carries hexadecimal rather than JSON:

```
AT+SEND=0001,140,<140 lowercase hex characters>
```

The radio’s AT command buffer (RADIO_AT_CMD_MAX_LEN), the inbound +RCV payload buffer (RADIO_RCV_MAX_LEN), and the generated artifact limit (PACKET_MAX_RCV_LEN) are all 256 bytes, which comfortably holds the largest possible envelope plus the AT and +RCV framing. The line accumulator is one byte larger than the command limit so it can hold the terminating NUL.

IV.I. Envelope on the Wire

The sealed envelope is the lowercase hexadecimal encoding of a fixed layout (see the Cryptographic Design section):

```
nonce[24] || ciphertext[L] || tag[16]
```

For a full 48-byte body this is $24 + 48 + 16 = 88$ bytes, or 176 hex characters plus a trailing NUL, for a 177-byte envelope buffer (ENVELOPE_MAX_HEX_LEN). A live reading body is 29 or 30 bytes, so the common on-wire envelope is 138 or 140 hex characters. The declared length L in the AT+SEND and +RCV framing is the length of this hex string, not of the underlying JSON.

IV.II. Declared-Length Slicing Invariant

Given the substring T after the second comma:

$$C = T[0 : L] \quad \text{and} \quad T[L] = ,$$

The invariant $T[L] = ,$ is checked, so a mismatch between the declared length and the actual payload is a parse error rather than silent corruption. This is what makes comma-bearing and hex-bearing payloads safe to carry.

V. DHT11 Edge Timing

The DHT11 response consists of 40 bits transmitted as high-pulse-width modulation. Each bit begins with a $50 \mu\text{s}$ low level, then a high pulse whose width encodes the bit value:

- logical zero: $26 \mu\text{s} \leq w < 50 \mu\text{s}$,
- logical one: $50 \mu\text{s} \leq w \leq 70 \mu\text{s}$.

The

decoder applies $\text{SENSOR_ONE_THRESHOLD_US} = 50$ as the class boundary and accumulates the five response bytes (humidity high, humidity low, temperature high, temperature low, checksum):

$$\text{checksum} = (b_0 + b_1 + b_2 + b_3) \bmod 256$$

with rejection on mismatch. A signed temperature is encoded with bit 7 of the temperature-high byte as a sign bit. Every edge wait carries a $240 \mu\text{s}$ timeout (SENSOR_EDGE_TIMEOUT_US), so a dead bus reports SENSOR_RESULT_TIMEOUT rather than hanging the monitor tick.

V.I. Waveform Testing

The native test suite drives a fake clock and a GPIO timeline: edges are encoded as absolute time/level pairs, and gpio_get derives the current level from the timeline. This covers the happy path as well as every timeout shape: a response low that never ends, a response high that never ends, a data-bit low that never rises, a high pulse that never falls, and a checksum failure from a corrupted final bit.

VI. Peripheral Set and Annunciation

The expanded peripheral set is not decorative. Each device reports or acts on the same physical state the telemetry describes.

The three status LEDs are the local face of the cold chain. A temperature above the breach ceiling (COLD_CHAIN_MONITOR_TEMP_BREACH_TENTHS, 0 tenths) lights the red LED; a value at or below the warning ceiling (COLD_CHAIN_MONITOR_TEMP_WARN_TENTHS, -50 tenths) with a valid reading lights green; a failed or out-of-band reading lights yellow. Exactly one lamp is driven at a time.

The acknowledge button on GP15 is an active-low input with an internal pull-up and a 30 ms debounce window. One press consumes one edge, prints ACK, and re-seals the damper, giving the operator a manual override of the last automatic actuator state.

A 1602 LCD is driven through a PCF8574 I2C backpack. Four-bit nibbles are sent with a full enable pulse, and

rendering emits DDRAM address commands 0x80 and 0xC0 followed by sixteen padded characters per row:

```
line 1: T:23.0C H:61.0%
line 2: N:07 S:0042 OK
```

Initialization runs a classic HD44780 software sequence (function set, display on, clear, entry mode) and shortens to a single failure probe that returns `false` when the backpack does not acknowledge, keeping the monitor init path testable. The onboard LED (GP25) blinks three times on a successful transmit and once on any inbound +RCV report.

VII. Infrared Control Surface and the Damper

The VS1838B is a 38 kHz demodulating infrared receiver. Its output idles high and pulls low during a mark, so the decoder times edges and reconstructs a NEC pulse train. NEC frames begin with a 9 ms leader mark (accepted in the 8 to 10 ms window), followed by 32 bits transmitted least-significant-bit first. Each bit carries a $560\ \mu\text{s}$ mark (accepted in the 400 to $800\ \mu\text{s}$ window) and a space of about $560\ \mu\text{s}$ for a logical zero or about $1690\ \mu\text{s}$ for a logical one; the decoder classifies a space at or below $900\ \mu\text{s}$ as zero and at or above $1400\ \mu\text{s}$ as one. Because the receiver is noise-prone, the decoder also rejects a frame whose command byte is not the bitwise complement of its inverse byte. At most 68 durations are captured per frame.

Two decoded commands actuate the node:

- `MONITOR_IR_VENT_COMMAND` (0x45) calls `servo_vent`, driving the damper to its vented position;
- `MONITOR_IR_SEAL_COMMAND` (0x46) calls `servo_seal`, driving it back to the sealed position.

The SG90 servo is driven by a 50 Hz PWM signal with a 500 to $2500\ \mu\text{s}$ pulse range, mapped linearly from 0 to 180 degrees. Sealed is 0 degrees; vented is 90 degrees. The 1000 uF bulk capacitor on the servo supply rail is what keeps a sudden damper move from browning out the Pico.

The physical stakes are the point. The servo has no way to know who issued the command. A universal remote, a captured command replayed later, or a forged frame that survives to the actuator path all move the same

damper. When the damper vents, the cold store warms; the LEDs and LCD can still report nominal because the actuator path and the sensing path are separate. This is the paper’s central defensive lesson: a trustworthy reading is not the same as a trustworthy command.

The infrared control surface is deliberately left unauthenticated in the lab. The LoRa surface, by contrast, is closed below.

VIII. Cryptographic Design

The radio is the first open door, and it is the one a key can close. The design goal is that a forged or modified telemetry frame must fail before any JSON is parsed or any actuator decision is made. Two primitives provide that property, and both are implemented in this repository with no third-party code.

VIII.I. Argon2id Key Derivation

A passphrase is not a key. Argon2id (RFC 9106) [8] is a memory-hard password hash that mixes the passphrase and a salt across memory and time so that recovering the field passphrase from a captured image is expensive. The node derives a 32-byte session key at initialization with the classroom profile $t=3$, $p=1$, $m=64$ blocks (`CRYPTO_KDF_TIME_COST`, `CRYPTO_KDF_PARALLELISM`, `CRYPTO_KDF_MEMORY_BLOCKS`). That profile is sized to fit the RP2350 SRAM budget; it is a teaching parameter, not a hardening parameter, and the documentation says so. The salt must be at least 8 bytes; the laboratory salt is the 16 ASCII bytes `coldiron-salt-01`.

The in-repo derivation is built from BLAKE2b and the Argon2 variable-length hash H' , and it is checked against the RFC 9106 test vector in the native suite (`test_rfc9106_argon2id_vector`).

VIII.II. XChaCha20-Poly1305 per Frame

Every telemetry frame is sealed with XChaCha20-Poly1305, an AEAD that combines the ChaCha20 stream cipher and the Poly1305 one-time authenticator from RFC 8439 [7] with an extended-nonce construction. The 24-byte nonce is expanded through HChaCha20 into a per-frame subkey, which yields two properties that matter here:

- **Unpredictable nonces at scale.** A 192-bit nonce may be drawn at random for every frame from the RP2350 hardware random source (`get_rand_32`), so the node never needs a shared counter that a reboot could reuse.
- **One pass for secrecy and integrity.** The same operation produces the ciphertext and a 128-bit Poly1305 tag. An attacker who guesses a valid tag succeeds with probability 2^{-128} .

The associated data is the node identifier, a single byte (0x07 for the default node). It is authenticated but not encrypted, so a frame sealed for one node cannot be silently relabeled as another node’s frame even though the payload itself does not carry the sender. Binding the identity into the tag is what converts the hub’s old “trust the sender field” model into a cryptographic check.

VIII.III. Why ChaCha20 over AES on the RP2350

The RP2350 does not have a hardware AES engine; its accelerated crypto block covers SHA-256, not AES. A software AES implementation on this part is therefore both slower and riskier: table-driven AES performs data-dependent memory accesses, and those accesses create a cache-timing side channel. ChaCha20 is built only from addition, rotation, and XOR, with no data-dependent table lookups, so it is fast in portable C and has no comparable cache-timing surface. XChaCha20-Poly1305 is thus both the modern choice and the pragmatic one for this silicon.

The primitives are split across small, independently testable modules: `src/chacha20.c` (ChaCha20 block function, stream, and HChaCha20 subkey derivation), `src/poly1305.c` (Poly1305), `src/crypto_aead.c` (XChaCha20-Poly1305 seal and open with constant-time tag comparison), `src/blake2b.c` (BLAKE2b and H’), `src/argon2.c` (Argon2id core with BLAMKA and hybrid addressing), `src/crypto_kdf.c` (the passphrase KDF), and `src/envelope.c` (the hex envelope codec). A constant-time comparison (`crypto_aead_tag_equal`) ensures a mismatching tag is rejected without an early-exit timing signal.

IX. Envelope Layout and Gateway Verification

The binary envelope is assembled in a fixed order and then hex-encoded:

```
envelope = nonce[24], ciphertext[L], tag[16]
```

The encoder emits lowercase hex with a trailing NUL, and the decoder accepts either case. It requires an even-length string of at least the nonce plus tag size, bounds the decoded length, recomputes the tag over the associated data and ciphertext, compares in constant time, and only then decrypts. Any malformed, truncated, tampered, or forged envelope returns false and yields no trusted plaintext.

On the gateway side, `scripts/gateway.py` mirrors the same construction in pure Python using the standard library and the `field_crypto` module. The processing order is deliberate:

1. Parse the +RCV line by declared length to recover the hex envelope.
2. Authenticate and open the envelope. If the tag does not verify, log the frame as UNAUTHENTICATED with an empty payload and stop. The forged JSON is never parsed.
3. Only for an authenticated frame, decode the JSON, write an OK row, and apply the freeze policy.

The CSV log therefore grows by one row per frame with columns `utc`, `sender`, `auth`, `length`, `payload`, `rsi_snr`, and the `auth` column is the audit trail. The spoofing client `scripts/spoof.py` holds no field key, so it cannot produce a valid envelope; its forged frames now appear only as rejected rows.

IX.I. Authenticated Threat Outcome

The attack that motivated the original monitor still runs, but the outcome flips. Under the unauthenticated protocol a forged frozen-temperature body was logged as a victim-origin frame and triggered the freeze actuator reply. Under the authenticated protocol the identical frame fails authentication, no JSON is parsed, no reply is sent, and the log records a rejection attributed to the claimed sender. The defense is observable on the same wire that carried the attack.

X. Artifact Contract

Provisioning constants are stored in a JSON artifact:

```
{
  "format": "cold-chain-monitor-packets-demo-v1",
  "frame_version": 1,
  "node_id": 7,
  "hub_address_hex": "0001",
  "frame_size": 48,
  "tx_interval_ms": 5000,
  "dht_timeout_us": 240,
  "lcd_i2c_address_hex": "27",
  "max_rcv_len": 256,
  "example_frame": "{ \"n\":7, \"s\":0, \"t\":235, \"h\":610}"
}
```

scripts/gen_packet.py emits include/packet_artifact.h from the JSON byte-for-byte. The CMake build regenerates the header before compiling and fails when the committed header is stale, so firmware constants and the test suite always read the same provisioning data. The receive limit is 256 bytes, matching the radio command and receive buffers so a maximum-size hex envelope fits with framing headroom.

XI. LCD Rendering

The 1602 LCD rendering path is described with the wider peripheral set in the Annunciation section. In short, line 1 shows the decoded reading and line 2 shows the node id, sequence number, and an OK or !! sensor status, and the module fails closed when the backpack does not acknowledge during initialization.

XII. Hub Gateway and Authenticated Actuator Reply

scripts/gateway.py first programs the hub radio (AT+ADDRESS=1, AT+NETWORKID=18) and then processes every +RCV line as described above. When an authenticated payload decodes to a temperature below the freeze threshold $F = -50$ tenths, the hub replies with an actuator command:

```
AT+SEND=<sender>,20,{"cmd":"heat","v":1}
```

The reply target is now trustworthy because it is only reached after the envelope authenticated. This preserves the original observable consequence for honest nodes while denying it to an attacker who cannot seal a frame. A production alert path would still add debouncing, hys-

teresis, and alarm health checks; the lab keeps the policy simple so the authentication result is unambiguous.

XIII. Attack Exercises and Hardening

The classroom runs the same two exercises against the authenticated build.

XIII.I. LoRa Sender Spoof

scripts/spoof.py reconfigures its own radio to the victim node address via AT+ADDRESS, then injects a forged frozen-temperature frame. It offers two modes. In plaintext mode it sends the bare JSON body that the old hub trusted. In bad-tag mode it wraps the forged body in a structurally plausible hex envelope with a random nonce and a random tag, which models an attacker who understands the format but holds no key. Both modes fail at the gateway: the first is not a valid envelope, and the second fails tag verification. This is a pedagogical reintroduction of a well-known LoRa failure mode: at the physical and MAC layer nothing binds a frame to a physical transceiver, so authentication must live in the payload.

XIII.II. NEC Infrared Replay

The infrared eye accepts any NEC frame. The exercise captures a vent command and replays it, and the damper moves. Because the command is not keyed and the receiver has no notion of freshness, replay succeeds indefinitely. This surface is documented rather than fixed in the lab build.

XIII.III. What the Hardening Buys, and What It Does Not

The authenticated envelope closes the telemetry surface:

- A forged or modified frame fails the tag before parsing.
- The node identity is bound into the associated data, so a frame cannot be relabeled for another node.
- The gateway's decision to actuate depends on authenticated data.

It does not, by itself, fix replay of a captured valid envelope, key compromise, or the infrared path. Those

require the additional controls listed in the Threat Model section.

XIV. Implementation Compliance Mapping

The repository implements the full classroom loop:

- **Peripherals and control:** `src/monitor.c` drives the tick and the actuator policy; `src/sensor.c` samples and formats; `src/display.c` renders; `src/status_led.c` maps temperature to the red, yellow, and green lamps; `src/button.c` debounces the acknowledge input; `src/servo.c` drives the damper PWM; `src/ir_remote.c` decodes NEC commands.
- **Radio:** `src/radio.c` provisions the transceiver (AT+ADDRESS, AT+NETWORKID), builds AT+SEND, parses +RCV with the declared-length discipline, and pumps CRLF lines into 256-byte buffers.
- **Cryptography:** `src/chacha20.c`, `src/poly1305.c`, `src/crypto_aead.c`, `src/blake2b.c`, `src/argon2.c`, `src/crypto_kdf.c`, and `src/envelope.c`, with `include/field_secrets.h` holding the lab-only key material.
- **Tooling:** `scripts/gen_packet.py`, `run_tests.py`, `check_coverage.py`, `audit_c_standard.py`, `audit_python_standard.py`.
- **Classroom:** `scripts/gateway.py` (hub provisioning, authentication, CSV logging, authenticated freeze reply), `scripts/spoof.py`, `scripts/sim_edge.py`, and the pure-Python interoperable crypto in `scripts/field_crypto.py`.
- **Tests:** 89 native C cases. They cover the full DHT waveform and every timeout shape, the declared-length parser with comma-bearing payloads, the NEC decoder and its malformed, ambiguous, and timeout shapes, the servo mapping, the LED mapping, the debounce logic, the monitor paths for transmit, in-bound-blink, sensor-timeout, infrared vent and seal, and missing-key, and the cryptographic primitives against published vectors and round-trip, tamper, and interoperability checks.

The tests run natively on the host via mock Pico SDK headers, reaching 100% line coverage on `crc.c`, `sensor.c`, `display.c`, `radio.c`, `status_led.c`, `button.c`, `servo.c`, `ir_remote.c`, `chacha20.c`, `poly1305.c`, `crypto_aead.c`, `blake2b.c`, `argon2.c`, `crypto_kdf.c`, `envelope.c`, and `monitor.c` under

LLVM source coverage. Two test hooks, `sensor_deinit` and `monitor_deinit`, exist solely to return those modules to their uninitialized policy states so the guard clauses are exercised.

XV. Threat Model and Limitations

The security claims of this build are bounded and stated plainly.

- **Lab key profile.** Argon2id runs at $t=3$, $p=1$, $m=64$ blocks so the derivation fits the RP2350 SRAM budget. This is weaker than a production password-hashing profile and must be raised on a host gateway.
- **Keys in flash are development-only.** `include/field_secrets.h` commits a shared passphrase and salt so that the firmware and the Python gateway derive the same key in the classroom. Production firmware must provision the session key from one-time-programmable (OTP) memory at manufacture and must never embed a passphrase, salt, or derived key in flash. Shipping this file as-is is a lab convenience, not a secure deployment.
- **Shared key across nodes.** The lab uses one field key for all nodes. A compromised node can therefore seal frames that authenticate for any node identity permitted by the associated data. Per-node keys or a key hierarchy would be required in the field.
- **No gateway sequence window.** The sequence number is inside the sealed body and is therefore authenticated, but the gateway does not yet enforce monotonicity across frames. A captured valid envelope can be replayed until a sequence ledger is added.
- **Infrared path unauthenticated.** The NEC receiver has no key and no anti-replay state. Any compatible remote can move the damper. This is a documented exercise, not a defended surface.
- **Sensor trust boundary.** The DHT11 is a slow, checksummed but not authenticated one-wire sensor; the sampled values are only as trustworthy as the physical wiring and the ADC timing. Authentication protects the frame, not the sensor.
- **RSSI and SNR are informational.** Neither is a reliable origin indicator.
- **Artifact guardrail.** The build-time artifact check verifies provisioning consistency, not security.
- **Denial of service.** An attacker on the band can still jam or flood the receiver; authentication is not availability.

XV.I. Future Work

- Provision the session key from RP2350 OTP memory and remove the committed lab secret from the shipping build.
- Add a hub-side authenticated sequence window and a small replay cache so a captured envelope is rejected on second use.
- Move to per-node keys, or derive the key with the node identifier as KDF context, so one compromised node does not endanger the fleet.
- Add an actuator-state ledger and alarm debounce to the gateway.
- Raise the Argon2id profile on the gateway and record the derivation cost as a measured parameter.
- Investigate a keyed infrared path, such as a challenge-response remote, or replace the optical surface with an authenticated wired service port.

XVI. Conclusion

OPERATION COLD IRON turns a truthful but trusting cold-chain monitor into a defensible one. The node drives the full Embedded Hacking peripheral set, so a data-integrity failure has a visible and physical consequence at the damper. The LoRa telemetry surface is sealed end to end with XChaCha20-Poly1305 keyed through Argon2id, implemented and tested entirely in-repo, with the node identifier bound as associated data. The hub gateway authenticates before it parses, so the spoofing client that once injected forged readings now fails at the tag and its JSON never becomes truth. The infrared surface is left open, named, and explained rather than hidden. The firmware, hub toolset, artifact guardrail, and 100%-line-covered native test suite provide a reproducible baseline, and the threat model states exactly which assumptions remain. That combination, a closed door next to an honest account of the one still open, is the lesson the cold chain needs.

XVII. References

[1] D-Robotics, “DHT11 Digital temperature and humidity sensor datasheet,” Aosong Electronics Co., Ltd, 2010.

[2] Anonymous the Security Researcher, “Analysis of serial-AT sub-GHz radios: cleartext configuration and absent frame authentication,” Embedded security working notes, 2022.

[3] R. Menon and A. Prakash, “On the (in)security of LoRa point-to-point links under address spoofing,” *ACM SIGCOMM Embedded Systems Workshop*, 2023, pp. 12-19.

- [4] M. H. Rahman, M. R. Hoque, and S. M. Rahman, “IoT-enabled cold chain monitoring: A survey,” *IEEE Access*, vol. 10, pp. 21461-21486, 2022.
- [5] LoRa Alliance, “LoRaWAN Application Layer Security Specification,” LoRa Alliance Technical Committee, v1.0.4, 2020.
- [6] K. Thomas, “The reverse engineering self-study course,” <https://github.com/mytechnotalent/Reverse-Engineering>, 2026.
- [7] Y. Nir and A. Langley, “ChaCha20 and Poly1305 for IETF Protocols,” RFC 8439, Internet Engineering Task Force, June 2018.
- [8] A. Biryukov, D. Dinu, D. Khovratovich, and S. Josefsson, “Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications,” RFC 9106, Internet Engineering Task Force, September 2021.